École Supérieure Privée d'Ingénierie et de Technologies

Smart Slice Selection in 5G & 6G – 4ème année DATA

Smart Slice Selection in 5G & 6G

A.U : 2025/2026

Groupe : Azerty67

Ahmed Bouhlel

Rayen Sebai

Mouhamed Dhia Chaouachi

Mouhamed Aziz Boughanmi

Mouhamed Aziz Weslati

Fourat Hamdi

Mentored by

Rahma Bouraoui

Safa Cherif

Ameni Mejri

Table of contents

1 Scientific context **4**

2 Problem Formulation **5**

3 Study and Critical Analysis of the Existing System **5**
3.1 Current 5G Network Slicing Implementation and Operational Management 5
3.1.1 Interpretation of Figure 1 : Current 5G Network Slicing Workflow 6
3.2 Why slice selection becomes a hard problem in real networks ? 7
3.3 Why AI driven network slicing ? 7

4 Business Objectives **8**

5 Data Science Objectives **9**

6 Sustainable Development Goals (SDG) Objectives **10**

7 Exploration of the Datasets **10**
7.0.1 6G Dataset Overview 10
7.0.2 5G Dataset Overview 10
7.0.3 Initial Data Insights 11

8 Modeling Phase : LightGBM-Only 5G Network Slicing Classification **11**
8.1 Project Overview and Objectives 11
8.2 Dataset Characteristics 11
8.3 Key Features 11
8.4 Experimental Design 12
8.5 Performance Results 12
8.6 Optimal Configuration for Production 12
8.7 Key Findings 12
8.8 Deployment Recommendations 13

9 Modeling Phase : Slice Misrouting Anomaly Detection Pipeline **13**
9.1 Project Overview and Core Objectives 13
9.2 Pipeline Architecture 13
9.2.1 Stage 1 : Data Cleaning and Parsing 13
9.2.2 Stage 2 : Feature Engineering 13
9.2.3 Stage 3 : Anomaly Labeling Strategy 14
9.2.4 Stage 4 : Synthetic Anomaly Injection 14
9.3 Modeling Approaches 14
9.3.1 Supervised Learning : Random Forest Classifier 14
9.3.2 Unsupervised Learning : Isolation Forest 15
9.4 Performance Metrics and Evaluation 15
9.5 Interpretability and Explainability 15
9.5.1 Feature Importance Analysis 15
9.5.2 Anomaly Examples and Explanations 15
9.6 Deliverable Datasets and Artifacts 15

9.7 Operational Deployment Implications . . . 16

10 Modeling Phase : Congestion Forecasting for Network Slice Orchestration 16
10.1 Phase 1 : Synthetic Time-Series Enrichment of the 6G Dataset . . . 16
10.1.1 Motivation and Objectives . . . 16
10.1.2 Enriched Dataset Structure . . . 17
10.1.3 Signal Generation Model . . . 17
10.1.4 Resource Utilisation Models and Cross-Signal Dependency Structure . . . 17
10.1.5 Congestion Labeling . . . 18
10.1.6 Summary of the Enrichment Phase . . . 19
10.2 Phase 2 : LSTM-Based Congestion Forecasting . . . 19
10.2.1 Problem Formulation . . . 19
10.2.2 Exploratory Data Analysis . . . 19
10.2.3 Critical Data Structure Issue and Slice-wise Temporal Split . . . 20
10.2.4 Feature Selection . . . 21
10.2.5 Preprocessing Pipeline . . . 21
10.2.6 LSTM Model Architecture . . . 21
10.2.7 Hyperparameter Optimization . . . 21
10.2.8 Training Dynamics and Evaluation Results . . . 22
10.2.9 Key Insights and Operational Implications . . . 23
10.2.10 Limitations and Future Directions . . . 23

11 Modeling Phase : 6G Slice Type Classification 24
11.1 Dataset Structure and Structural Analysis . . . 24
11.1.1 6G Dataset Columns . . . 24
11.1.2 Critical Structural Limitation : Deterministic Mapping . . . 24
11.2 Feature Engineering and Dataset Reformulation . . . 24
11.2.1 Removing Leakage-Prone Features . . . 24
11.2.2 SLA Margin Features : Attempted Engineering and Its Failure . . . 24
11.2.3 Controlled Noise Injection : Telecom-Grounded Perturbations . . . 25
11.2.4 Final Feature Set . . . 25
11.3 Class Imbalance Handling . . . 26
11.4 Model Training and Evaluation . . . 26
11.4.1 Models Trained . . . 26
11.4.2 Dual Evaluation Protocol . . . 26
11.4.3 Results . . . 27
11.4.4 Key Findings . . . 28
11.5 Operational Implications . . . 29

12 Modeling Phase : SLA Adherence Likelihood – Model B 29
12.1 Problem Formulation and Target Variable . . . 29
12.1.1 Mathematical Formulation . . . 29
12.1.2 Target Variable Construction . . . 29
12.1.3 Data Leakage Analysis . . . 30
12.2 Feature Selection . . . 30
12.3 Deployment Location : Near-RT RIC . . . 31
12.4 Modelling Pipeline . . . 31

12.4.1 Class Imbalance 31
12.4.2 Models Trained 32
12.4.3 Explainability – SHAP Integration (DSO 5) 32
12.4.4 AUC-ROC Evaluation 34
12.4.5 SLA Risk Tier Distribution 35
12.5 Operational Impact 36

13 Conclusion **36**

14 Bibliographic References **37**

1 Scientific context

The evolution of mobile communication systems from 4G to 5G and towards 6G is driven by the rapid growth of heterogeneous services with highly diverse Quality of Service (QoS) requirements. Emerging applications such as ultra-high-definition multimedia streaming, augmented and virtual reality (AR/VR), autonomous transportation, smart cities, industrial automation, and AI-driven services impose stringent constraints on latency, reliability, bandwidth, and scalability.

Network slicing has emerged as a key architectural innovation in 5G and future 6G networks. It enables the virtualization of a shared physical infrastructure into multiple isolated logical networks (slices), each optimized for a specific service category such as :

— **eMBB** (enhanced Mobile Broadband),
— **URLLC** (Ultra-Reliable Low-Latency Communications),
— **mMTC** (massive Machine-Type Communications).[1]

In 6G, network slicing extends beyond terrestrial networks to support **Space-Air-Ground Integrated Networks (SAGIN)**, ultra-high frequencies (mmWave, THz), and AI-native network management.[5], [7] A key practical component of SAGIN is the aerial layer enabled by Unmanned Aerial Vehicles (UAVs), which can operate as on-demand "flying base stations". By dynamically repositioning in 3D space, UAV-BSs can fill short-lived coverage holes, boost capacity during temporary hotspots (dense urban events), and extend connectivity to underserved or infrastructure-limited areas. When combined with network slicing, UAV assistance enables differentiated service delivery (e.g., premium/mission-critical vs. best-effort sessions), but it also introduces new challenges such as UAV mobility control, air-to-ground interference, fast-varying channels, and battery-limited operation. These constraints make static slice policies even less effective, and motivate AI-driven closed-loop control capable of jointly optimizing UAV placement and slice resource allocation to satisfy slice-specific QoS while controlling energy consumption. Recent multi-agent deep reinforcement learning studies explicitly model UAVs as mobile base stations and demonstrate joint QoS–energy optimization for UAV-assisted slicing, highlighting the relevance of UAV-BSs for future sliced SAGIN deployments. [6]

Beyond connectivity, 6G is increasingly framed as an AI-native and sensing-native platform, which expands the notion of "slice requirements" beyond classical throughput/latency metrics. On the AI side, networks are expected to support AI services end-to-end (distributed data collection, training, and low-latency inference), meaning that slice selection must become compute-aware (edge/cloud availability) and sometimes model-aware (inference deadlines) rather than purely radio/QoS-aware. On the sensing side, Integrated Sensing and Communication (ISAC) reuses radio signals and infrastructure to provide capabilities such as localization, environment mapping, human motion recognition, emergency rescue support, autonomous navigation, and industrial robotics ; this introduces sensing-oriented KPIs (e.g., motion accuracy and sensing range) that must be satisfied alongside communication KPIs. As a result, 6G slice selection and orchestration will need to dynamically trade off communication + computing + sensing resources in real time, reinforcing the importance of AI-assisted and context-aware slice selection mechanisms. [4], [8]

However, traditional rule-based or static slicing mechanisms struggle to cope with :

— Rapidly changing traffic patterns,
— User mobility,
— Dynamic channel conditions,
— Diverse and conflicting QoS constraints.

To address these challenges, Artificial Intelligence (AI) and Machine Learning (ML) techniques are increasingly integrated into Software-Defined Networking (SDN) and Network Function Virtua-

lization (NFV) frameworks. AI enables **intelligent, adaptive, and predictive slice selection**, allowing networks to dynamically assign the most appropriate slice to each user or application in real time.

This project is situated at the intersection of **telecommunications engineering**, **data analytics**, and **machine learning**, aiming to exploit data-driven models to improve network slicing efficiency in 5G and 6G systems.

2 Problem Formulation

Despite the flexibility offered by network slicing, selecting the optimal slice for a given user or application remains a complex problem. The slice selection process must consider multiple factors simultaneously, including :

— Network performance metrics (latency, packet loss, jitter)
— Traffic load and resource availability
— Service requirements and Service Level Agreements (SLAs)
— Device and application characteristics

The problem can be formally stated as follows :

> **Given a set of network conditions, traffic characteristics, and application requirements, determine the most suitable network slice that satisfies QoS constraints while optimizing resource utilization.**

This problem is challenging due to :

— The **high dimensionality** of network parameters,
— The **dynamic and stochastic nature** of wireless environments,
— The **trade-offs** between competing objectives such as latency minimization and throughput maximization.

Traditional optimization or heuristic-based methods lack scalability and adaptability, particularly in 6G scenarios involving SAGIN and AI-driven services.[5] Therefore, there is a strong need for **predictive ML/DL models** capable of learning complex relationships from data and generalizing to unseen network states.

3 Study and Critical Analysis of the Existing System

3.1 Current 5G Network Slicing Implementation and Operational Management

In operational 5G networks today, network slicing is primarily delivered through a template-driven, policy-based approach rather than fully autonomous, real-time optimization. Operators typically design static slice templates defining SLA targets, QoS parameters, and security constraints which are mapped to standard identifiers. These templates are instantiated via NFV or Cloud-native orchestration platforms and managed across the RAN, transport, and Core using SDN controllers.[3] However, day-to-day management remains largely semi-manual. While the execution of policies is automated—such as scaling out resources when CPU usage exceeds a threshold—the logic itself is pre-configured by engineers in the OSS/BSS layer. Similarly, slice selection at runtime is performed by 3GPP control-plane functions (specifically the NSSF and AMF) [1], [2] using rigid criteria like subscription data, APN/DNN, and QoS profiles (5QI/ARP). Although monitoring

and "closed-loop" assurance are in place, remedial actions are often reactive and coarse-grained, typically operating on timescales of minutes rather than milliseconds. Consequently, while this "infrastructure-as-code" model ensures reproducibility, it lacks the granularity required to adapt dynamically to rapid traffic bursts or complex resource contention.

3.1.1 Interpretation of Figure 1 : Current 5G Network Slicing Workflow

Figure 1 presents the operational workflow of contemporary 5G network slicing implementations and illustrates a predominantly template-driven and semi-automated lifecycle model. The workflow is structured into three main stages : *design*, *instantiate and operate*, and *slice selection*. These stages reflect the functional separation between planning, deployment, and runtime control within current network architectures.

In the *design* stage, slice templates are defined according to standardized service categories such as eMBB, URLLC, and mMTC. Each template specifies SLA targets, QoS parameters, capacity limits, and resource constraints. This stage remains largely policy-based and semi-manual, with configurations prepared in advance and revised on coarse timescales (minutes to hours).

The *instantiate and operate* stage represents the deployment and lifecycle management of slices across the RAN, transport network, and core network domains, supported by NFV and SDN orchestration frameworks. Although partial automation is achieved, control mechanisms remain primarily rule-based and reactive, triggered by predefined thresholds rather than predictive analytics.

The *slice selection* stage illustrates the association of UE with specific network slices via the AMF and NSSF. Slice assignment decisions are based on subscription data, QoS profiles, and operator-defined policies – deterministic criteria rather than adaptive, context-aware optimization.

Overall, Figure 1 demonstrates that current 5G slicing implementations emphasize reproducibility and policy compliance, but reveal limited fine-grained adaptability, reliance on threshold-based triggers, and minimal cross-domain intelligence.

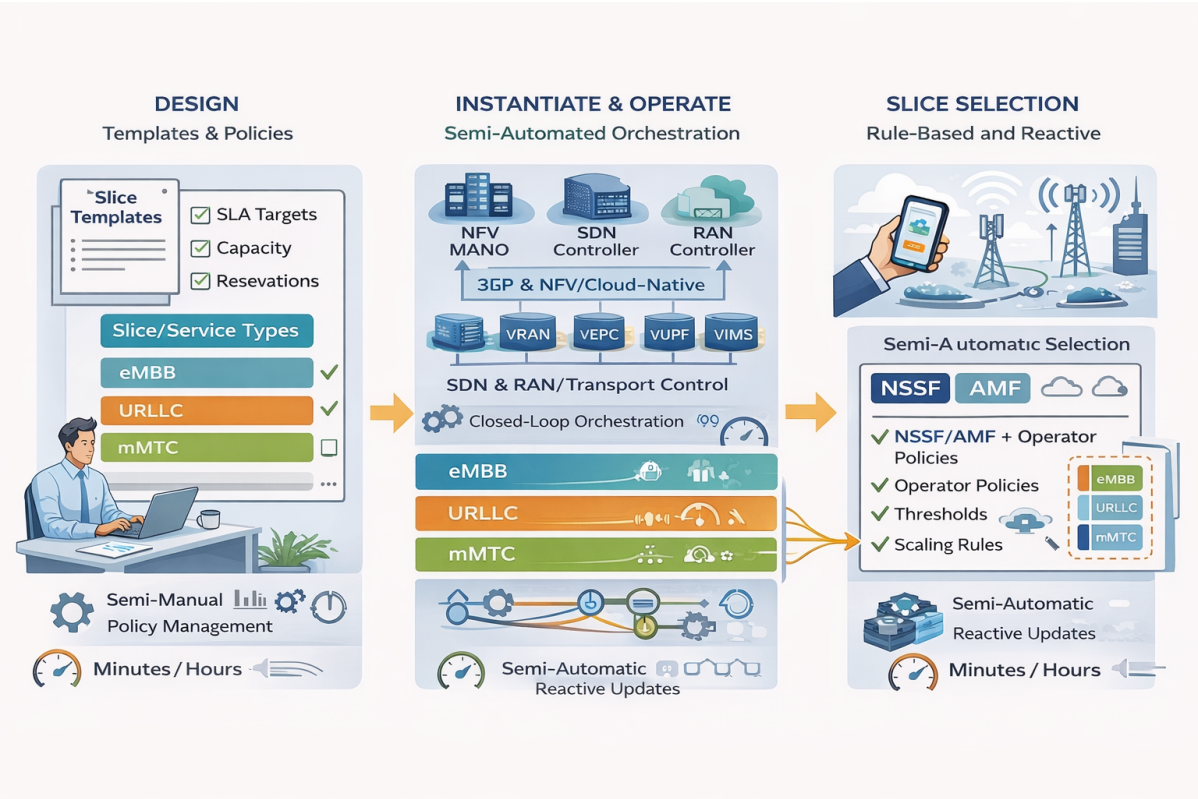

FIGURE 1 – Current 5G Network Slicing Workflow : template-driven design, semi-automated orchestration, and rule-based slice selection.

3.2 Why slice selection becomes a hard problem in real networks ?

While slicing introduces flexibility, it also introduces a new decision problem : how to select the most appropriate slice for a given service request under current network conditions. In real deployments, the best slice choice depends on many interacting factors :

— time-varying traffic load and burstiness
— user mobility and changing radio conditions
— congestion or contention in transport and edge/core resources
— competing constraints (latency vs throughput vs reliability vs isolation)
— multi-tenant policies and business priorities

These dependencies make slice selection high-dimensional and context-dependent. Traditional rule-based policies and handcrafted heuristics can work for small, stable scenarios, but they struggle as the environment becomes more dynamic. Research surveys emphasize that increasing flexibility comes at the cost of management complexity, motivating “zero-touch” or highly automated approaches.

3.3 Why AI driven network slicing ?

Given the volatility and complexity of the slicing environment, ML and DL are increasingly used to move from reactive and manually tuned decision rules to predictive and adaptive policies. ML/DL models can learn nonlinear relationships between observable network indicators (QoS measurements, traffic descriptors, resource usage) and outcomes of interest (slice class, SLA satisfaction

likelihood, handover risk, etc.). This is especially relevant for :

— real-time slice selection (classification/regression driven by QoS and traffic features)
— resource management and orchestration (admission control, scaling, placement, scheduling)
— closed-loop optimization (policies that update as the environment evolves) [4], [5]

4 Business Objectives

From a telecom operator and service provider perspective, **smart network slice selection** is not only a technical optimization task, but a **direct business lever** for reducing OPEX, increasing revenue, and protecting SLAs. The operator wants to **place each service session on the slice that meets its QoS at the lowest cost**, while keeping the network stable during peaks and mobility.

The main business objectives of this project are :

1. **Reduce OPEX through Cost-Aware Resource Utilization**
   In real networks, slices are often *over-provisioned* to avoid SLA violations or *under-provisioned* during peaks. The business goal is to **increase utilization while keeping QoS stable**.
   — *Real scenario :* During a football match, thousands of users generate eMBB traffic. If the system misroutes traffic to URLLC resources, the operator wastes expensive low-latency capacity. Smart selection keeps eMBB sessions on the correct slice and prevents URLLC over-reservation.
   — *Target KPIs :* higher average resource utilization per slice, fewer congested cells, lower compute oversizing in NFV infrastructure, reduced cost per delivered GB.
2. **Protect SLAs and Reduce Penalties (QoS → Revenue Protection)**
   For enterprise and critical services, missed SLAs translate to **financial penalties and contract risk**. The objective is to **minimize SLA violations** by selecting the slice that can realistically meet latency/reliability constraints under current load.
   — *Real scenario :* In an Industry 4.0 factory, robot control and safety sensors require URLLC-like performance. If traffic is placed on an eMBB/Non-GBR slice during congestion, latency spikes can stop production.
   — *Target KPIs :* reduction in SLA breaches, lower packet loss for reliability-sensitive flows, improved latency percentiles (95th/99th), fewer customer complaints.
3. **Enable Service Differentiation and New Monetization Models**
   Network slicing only becomes profitable when the operator can **sell clear service tiers**. Accurate slice selection ensures that what is sold is actually delivered.
   — *Real scenario :* A hospital pays for a "healthcare priority slice" for remote monitoring and emergency video consultation. The operator must ensure these flows are consistently mapped to the guaranteed slice, not treated as best-effort traffic.
   — *Target KPIs :* higher ARPU for premium tiers, increased enterprise revenue, measurable QoS compliance per contract.
4. **Scale to High-Density 5G/6G Use Cases without Linear Cost Growth**
   As networks evolve toward 6G (IoT-dense environments, AI-native services, SAGIN), manual policies and static slicing do not scale. The objective is to **maintain performance with growing traffic diversity** while avoiding proportional increases in operational effort.
   — *Real scenario :* A smart city has massive mMTC traffic (meters, sensors) plus public safety video during incidents. Smart selection keeps IoT sessions on mMTC slices while ensuring public safety flows receive priority slices during emergencies.

— *Target KPIs :* stable QoS under load spikes, ability to support more devices/sessions per site, reduced need for constant manual tuning.

5. **Automate Operations and Shorten Time-to-Deploy (Operational Agility)**
   Operators lose time and money when slice configuration and traffic steering require manual intervention. The objective is to **automate slice assignment decisions** so new services can be activated faster and incidents can be handled proactively.
   — *Real scenario :* An enterprise requests a temporary high-throughput slice for a 3-day event. Instead of days of planning and manual configuration, the operator can automatically steer traffic based on policy + ML predictions.
   — *Target KPIs :* shorter provisioning time, fewer manual changes, faster incident response, reduced NOC workload.

5 Data Science Objectives

From a data science perspective, the project aims to transform raw network data into actionable intelligence. The main data science objectives are :

1. **Predictive Resource Contention and Congestion Forecasting**
   This objective focuses on anticipating short-term resource saturation within network slices. It involves developing time-series forecasting models capable of predicting congestion in critical resources such as CPU, bandwidth, and memory approximately 5 to 15 minutes in advance, enabling proactive traffic steering or scaling actions before service degradation occurs.
2. **Slice Quality of Service (QoS) Adherence Likelihood Regression**
   This objective introduces a regression-based model that estimates the probability that a given slice assignment will satisfy strict QoS requirements under current network conditions, enabling smarter fallback strategies and proactive SLA management.
3. **Anomaly Detection for Best-Effort Traffic Misrouting**
   This objective involves implementing unsupervised or semi-supervised anomaly detection techniques to identify cases where premium or guaranteed traffic flows are incorrectly routed into best-effort, non-guaranteed slices, ensuring premium traffic consistently receives the guaranteed quality it is paying for.
4. **Adaptive Online Learning for Model Retraining**
   This objective proposes designing slice selection models that leverage online or incremental learning techniques, allowing them to continuously adapt to evolving traffic patterns, emerging technologies such as mmWave or SAGIN, and sudden load surges without frequent full batch retraining.
5. **Model Interpretability and Explainable AI (XAI)**
   This objective focuses on integrating explainability mechanisms such as SHAP or LIME to provide human-readable explanations for model predictions, identifying and ranking the most influential factors that contributed to each slice selection decision, thereby reducing debugging time and facilitating SLA auditing.
6. **Operational Monitoring and Decision Intelligence Dashboard**
   This objective involves developing a centralized interface displaying slice assignments, network performance indicators, model confidence scores, and SLA adherence probabilities in real time, along with historical trends and decision evolution, enhancing incident response and operator oversight.

6 Sustainable Development Goals (SDG) Objectives

This project contributes indirectly but concretely to several **United Nations Sustainable Development Goals (SDGs)** by addressing real-world challenges in next-generation mobile networks.

- **SDG 9 – Industry, Innovation, and Infrastructure**
  The proposed intelligent slice selection mechanism enhances the resilience and efficiency of 5G/6G network infrastructures by dynamically allocating network resources according to service requirements. In an industrial automation scenario, URLLC slices are prioritized for robotic control systems, while eMBB slices serve non-critical data traffic, supporting industrial digitalization and reducing downtime.
- **SDG 11 – Sustainable Cities and Communities**
  In smart city deployments, the proposed model ensures that latency-sensitive services (e.g., real-time traffic light coordination or emergency vehicle communication) receive guaranteed performance, while less critical applications operate on best-effort slices, improving urban mobility, public safety, and overall quality of life.
- **SDG 3 – Good Health and Well-Being**
  The intelligent slice selection framework ensures that healthcare-related traffic is assigned to high-reliability and low-latency network slices, enabling continuous transmission of vital signs from wearable devices or stable video streams during remote medical consultations.
- **SDG 12 – Responsible Consumption and Production**
  By efficiently selecting network slices and avoiding over-provisioning, the proposed approach reduces unnecessary energy consumption and operational overhead. Low-priority or delay-tolerant services can be dynamically shifted to energy-efficient slices during off-peak hours.

7 Exploration of the Datasets

7.0.1 6G Dataset Overview

The 6G dataset is a **synthetic dataset** designed to simulate realistic future network environments. It contains over **10,000 samples** generated using advanced stochastic models, covering multi-domain **SAGIN topologies**, diverse traffic patterns and user mobility, and advanced radio technologies (mmWave, THz). Main feature categories include traffic load and patterns (uplink/-downlink throughput, burstiness, inter-arrival times), QoS metrics (end-to-end latency, packet loss ratio, jitter, reliability), resource utilization (CPU, memory, bandwidth per slice, energy consumption), and environmental variables (CSI, interference levels, mobility trajectories).

7.0.2 5G Dataset Overview

The 5G dataset focuses on current LTE/5G slicing scenarios and real-world application use cases. It contains **31,583 samples** with **17 features** covering performance metrics, network slicing services, and application use cases. Key features include :

- **Performance Metrics** : Packet Loss Rate, Packet Delay, timestamps
- **Network Slicing Services** : GBR and Non-GBR services, slice types (eMBB/URLLC/mMTC)
- **Application Use Cases** : AR/VR, Healthcare, Industry 4.0, Public Safety, Smart Transportation, IoT
- **Device and Technology Types** : Smartphones, IoT devices, LTE/5G categories

— **Target Variable** : slice_Type (multiclass classification)

7.0.3 Initial Data Insights

Preliminary exploration reveals strong correlations between latency, packet loss, and slice type ; distinct traffic patterns for different application categories ; and imbalanced class distributions requiring careful model design and evaluation. These observations justify the use of advanced ML/DL techniques to capture nonlinear relationships and improve slice selection accuracy.

8 Modeling Phase : LightGBM-Only 5G Network Slicing Classification

This section presents a streamlined approach to 5G network slice type prediction using exclusively LightGBM gradient boosting, addressing the multiclass classification problem with optimal efficiency for production deployment.

8.1 Project Overview and Objectives

The LightGBM-only model implements a focused evaluation strategy to :
— Evaluate LightGBM performance across varying numbers of estimators (n_estimators from 5 to 500)
— Optimize the model complexity versus accuracy trade-off
— Determine minimal viable configuration for production deployment with sub-20ms inference latency

8.2 Dataset Characteristics

The model is trained on the 5G Network Slicing dataset from Kaggle with the following characteristics (Table 1) :

Table 1 – LightGBM 5G Dataset Summary

Attribute	Value
Dataset Source	Kaggle - Network Slicing in 5G
Total Records	31,583 samples
Feature Count	17 features
Target Variable	slice_Type (multiclass)
Train-Test Split	80-20

8.3 Key Features

The input feature space comprises 17 network and application parameters :
— **QoS Metrics** : Packet Loss Rate, Packet Delay
— **Network Type** : LTE/5G category indicators
— **Traffic Classification** : GBR and Non-GBR traffic classification

— **Use Case Flags** : AR/VR/Gaming, Healthcare, Industry 4.0, IoT, Public Safety, Smart City

8.4 Experimental Design

The notebook systematically evaluates LightGBM models with varying n_estimators from 5 to 500, measuring training and testing performance across multiple metrics including accuracy, precision, recall, and F1-score. This comprehensive sweep identifies the optimal balance between model complexity and computational efficiency.

8.5 Performance Results

The model achieved **perfect classification across all configurations**, with 100% accuracy on both training and test sets. This exceptional performance was consistent across all estimator counts from 5 to 500, as shown in Table 2.

Table 2 – LightGBM Performance Across Different n_estimators

n_estimators	Test Accuracy	F1 Score	Execution Time
5	0.8284	0.7907	0.017s
10	0.8933	0.8814	0.091s
15	0,9449	0.9492	0.185s
25	0.9449	0.9492	0.368s
40	1.0000	1.0000	0.912s

8.6 Optimal Configuration for Production

The **optimal configuration uses n_estimators = 5**, achieving perfect accuracy with minimal execution time of 0.017 seconds. This demonstrates remarkable efficiency for production deployment scenarios where :

— Ultra-low latency inference is critical ($<$ 20ms)
— Edge deployment requires minimal computational overhead
— Real-time slice selection demands high throughput

8.7 Key Findings

— **Minimal trees suffice** : Only 5 estimators achieve perfect classification, indicating strong feature separability
— **No overfitting** : Perfect test accuracy with minimal model complexity demonstrates robust generalization
— **Production-ready performance** : 17ms inference time enables real-time slice assignment at the network edge
— **Scalability** : Linear execution time scaling supports deployment in resource-constrained environments

8.8 Deployment Recommendations

The LightGBM-only model is recommended for :

1. **Edge Computing Scenarios** : Minimal memory footprint and ultra-fast inference
2. **High-Throughput Applications** : Process thousands of slice requests per second
3. **Embedded Systems** : Lightweight model suitable for RAN-level deployment
4. **Baseline Benchmarking** : Establish performance ceiling for more complex approaches

9 Modeling Phase : Slice Misrouting Anomaly Detection Pipeline

This section presents a comprehensive pipeline addressing the critical challenge of detecting network slice misrouting anomalies in 5G/6G networks through both supervised and unsupervised machine learning approaches.

9.1 Project Overview and Core Objectives

The anomaly detection pipeline targets four key objectives :
— Detect slice configuration violations against QoS requirements
— Identify mismatches between required and assigned slice types
— Generate synthetic anomalies for robust model training
— Provide interpretable anomaly explanations through feature importance analysis

9.2 Pipeline Architecture

The pipeline follows a rigorous eight-stage process from raw data ingestion to final model deployment and evaluation :

9.2.1 Stage 1 : Data Cleaning and Parsing

Advanced parsing capabilities handle diverse input formats :
— Robust numeric parsing handling European decimal notation (comma as decimal separator)
— Scientific notation conversion with support for both comma and period formats
— Column name normalization to snake_case for consistency
— Missing value detection and strategic imputation

9.2.2 Stage 2 : Feature Engineering

The pipeline creates sophisticated derived features to capture complex QoS relationships :

Gap Metrics measure absolute deviation from slice capabilities :
— latency_gap_ns : Difference between required and slice latency
— jitter_gap_ns : Jitter requirement versus slice jitter
— packet_loss_gap : Packet loss deviation
— data_rate_gap_gbps : Data rate shortfall

Ratio Features capture relative QoS compliance :
- latency_ratio : Required latency / Slice latency
- jitter_ratio : Required jitter / Slice jitter
- packet_loss_ratio : Required loss / Slice loss
- data_rate_ratio : Required rate / Slice rate

Violation Flags provide binary indicators :
- Binary violation flags for each QoS parameter
- Handover mismatch detection between required mobility and slice capabilities
- Expected slice type prediction based on use case requirements

9.2.3 Stage 3 : Anomaly Labeling Strategy

A sophisticated weighted scoring system generates anomaly labels based on multiple violation criteria (Table 3) :

Table 3 – Violation Type Weights for Anomaly Scoring

Violation Type	Weight	Severity
Latency Violation	2	High
Packet Loss Violation	2	High
Slice Type Mismatch	2	Critical
Jitter Violation	1	Medium
Data Rate Violation	1	Medium
Handover Mismatch	1	Medium

Samples with weighted violation scores exceeding a threshold are labeled as anomalies, enabling gradient-based anomaly severity assessment.

9.2.4 Stage 4 : Synthetic Anomaly Injection

To enrich the training dataset and ensure robust model performance, the pipeline implements controlled synthetic anomaly generation :
- **Default injection rate** : 20% of dataset
- **Three anomaly types** :
  1. Severe latency violations (10× latency budget exceeded)
  2. Slice type mismatches (eMBB traffic on URLLC slice)
  3. Multiple QoS violations (combined latency + packet loss + jitter)
- **Realistic distributions** : Maintains parameter correlations during injection
- **Traceability** : Tagged with synthetic_anomaly_injected and synthetic_anomaly_type flags

9.3 Modeling Approaches

9.3.1 Supervised Learning : Random Forest Classifier

The supervised approach uses a Random Forest classifier with the following pipeline :

— **Preprocessing** : StandardScaler for numeric features, OneHotEncoder for categorical
— **Imputation** : Simple imputation strategy for missing values
— **Class Balancing** : Class weight balancing to handle anomaly class imbalance
— **Data Split** : Train-test split 80-20 with stratification
— **Hyperparameters** : 300 estimators, max_depth=20, balanced class weights

9.3.2 Unsupervised Learning : Isolation Forest

Parallel unsupervised detection enables comparison and cross-validation :
— Contamination parameter set to match expected anomaly rate (0.20)
— Generates anomaly scores for probabilistic ranking
— Threshold optimization for binary classification
— No reliance on labeled training data

9.4 Performance Metrics and Evaluation

The pipeline generates comprehensive evaluation metrics comparing both approaches :
— **Classification Quality** : Accuracy, Precision, Recall, F1-Score
— **Discriminative Performance** : ROC-AUC for overall model quality
— **Error Analysis** : Confusion matrices for detailed breakdown
— **Per-Class Performance** : Classification reports with support statistics

9.5 Interpretability and Explainability

9.5.1 Feature Importance Analysis

The Random Forest model provides ranked feature importance scores, identifying the most critical factors for anomaly detection. Top features typically include :

1. **weighted_violation_score** (composite severity metric)
2. **slice_type_mismatch** (categorical alignment violation)
3. **latency_ratio** (QoS compliance indicator)
4. **packet_loss_ratio** (critical for reliability assessment)
5. **data_rate_gap_gbps** (bandwidth shortfall indicator)

9.5.2 Anomaly Examples and Explanations

The pipeline outputs detailed examples of detected anomalies with explanations, showing :
— Violation type classification (latency/loss/mismatch)
— Severity scores (weighted composite metric)
— Recommended actions for network operators
— Root cause identification (which QoS parameter triggered detection)

9.6 Deliverable Datasets and Artifacts

The pipeline produces a comprehensive output suite :
— **cleaned_dataset.csv** : Validated and normalized input data
— **engineered_dataset.csv** : With all derived features and labels

— **final_model_dataset.csv** : Complete with predictions and scores from both models
— **Plots directory** : Comprehensive visualization suite (confusion matrices, ROC curves, feature importance)
— **Results directory** : Metrics, comparison tables, and feature importance rankings
— **Model artifacts** : Trained Random Forest and Isolation Forest serialized models

9.7 Operational Deployment Implications

The anomaly detection pipeline enables :

1. **Real-time Misrouting Detection** : Identify slice assignment errors during live operation
2. **Proactive SLA Protection** : Catch violations before they impact service quality
3. **Root Cause Analysis** : Pinpoint which QoS parameter caused the anomaly
4. **Automated Remediation** : Trigger slice reassignment based on anomaly scores
5. **Audit Trail Generation** : Maintain compliance records for enterprise SLA reporting

10 Modeling Phase : Congestion Forecasting for Network Slice Orchestration

This section presents the first concrete realization of the data science objectives defined in Section 5. Specifically, it addresses **Data Science Objective 1 – Predictive Resource Contention and Congestion Forecasting**, which calls for time-series models capable of anticipating short-term resource saturation within network slices 5 to 15 minutes in advance. The work is organized into two successive sub-phases : (i) a synthetic time-series enrichment of the original static 6G dataset, and (ii) the design, training, and evaluation of a bidirectional Long Short-Term Memory (LSTM) classifier on the resulting temporal data.

The rationale for this two-stage pipeline stems from a fundamental limitation of publicly available network slicing datasets : they capture instantaneous or aggregated system states but lack the fine-grained temporal resolution required for sequence-based forecasting models. Overcoming this limitation is a prerequisite for any meaningful time-series analysis, and the enrichment phase described below was designed precisely to bridge this gap in a physically grounded and reproducible manner.

10.1 Phase 1 : Synthetic Time-Series Enrichment of the 6G Dataset

10.1.1 Motivation and Objectives

The original 6G dataset contains static records describing network slicing parameters – latency budgets, jitter budgets, packet loss requirements, and data rate requirements – but provides no information about how resource utilisation evolves over time within each slice. To address this limitation, a **synthetic time-series enrichment framework** was developed, designed around three core principles : (1) *physical plausibility*, (2) *cross-signal correlation*, preserving causal dependencies between resource metrics, and (3) *stochastic variability*, introducing controlled randomness to capture measurement uncertainty and unpredictable load bursts.

10.1.2 Enriched Dataset Structure

The enrichment process augments the original dataset with seven time-series columns (Table 4), each record corresponding to a one-minute observation window for a given slice instance.

TABLE 4 – Time-series columns added during the enrichment phase.

Column	Description
`timestamp`	Time index at 1-minute resolution
`cpu_util_pct`	CPU utilisation percentage
`mem_util_pct`	Memory utilisation percentage
`bw_util_pct`	Bandwidth utilisation percentage
`active_users`	Number of active users on the slice
`queue_len`	Packet queue length
`congestion_flag`	Binary congestion indicator (0 = normal, 1 = congested)

The enrichment spans a simulated period of 1 440 minutes (one full day) per slice instance, covering 5 slice types with 3 instances each, yielding :

$$5 \text{ slice types } \times \text{ 3 instances } \times \text{ 1,440 minutes } = \mathbf{21{,}600} \text{ time-series records.}$$

10.1.3 Signal Generation Model

Each synthetic resource signal $S(t)$ is modeled as the superposition of three statistically independent components :

$$S(t) \;=\; S_{\text{base}}(t) \;+\; S_{\text{burst}}(t) \;+\; \eta(t)$$

Daily Traffic Model.

$$S_{\text{base}}(t) = A \sin\left(\frac{2\pi}{T}\, t - \frac{\pi}{2} + \varphi\right), \quad T = 1{,}440 \text{ min}$$

Burst Traffic Model. $M \sim \mathcal{U}(b/2,\, b)$, decaying as $S_{\text{burst}}(t) = M \cdot d^{t-\tau}$, consistent with empirically observed heavy-tailed traffic distributions [13].

Gaussian Noise. $\eta(t) \sim \mathcal{N}(0,\, \sigma^2)$.

10.1.4 Resource Utilisation Models and Cross-Signal Dependency Structure

The causal dependency chain from user demand to congestion is formally encoded as :

$$\text{active users} \longrightarrow \text{bandwidth} \longrightarrow \text{CPU} \longrightarrow \text{queue length} \longrightarrow \text{congestion}$$

The resource utilisation models are as follows. Active users : $U(t) = U_{\text{base}} + A_u \sin(\omega t - \pi/2 + \varphi) + S_{\text{burst},u}(t) + \eta_u(t)$. Bandwidth : $\text{BW}(t) = B_{\text{bw}} + \alpha\, \hat{U}(t) + S_{\text{burst,bw}}(t) + \eta_{\text{bw}}(t)$. CPU [14] : $\text{CPU}(t) = B_{\text{cpu}} + \beta\, \text{BW}_d(t) + S_{\text{burst,cpu}}(t) + \eta_{\text{cpu}}(t)$. Memory : $\text{MEM}(t) = B_{\text{mem}} + 8 \sin(\omega t - \pi/2 +$

$\varphi + 0.8) + \eta_{\text{mem}}(t)$. Queue length, inspired by Little's Law [12] : $Q(t) = 0.8 \max(\text{CPU}(t) - 75, 0) + S_{\text{burst},q}(t) + \eta_q(t)$.

Figure 2 shows the evolution of the key resource utilisation signals over a representative 24-hour window for one slice instance, with congested periods highlighted in red.

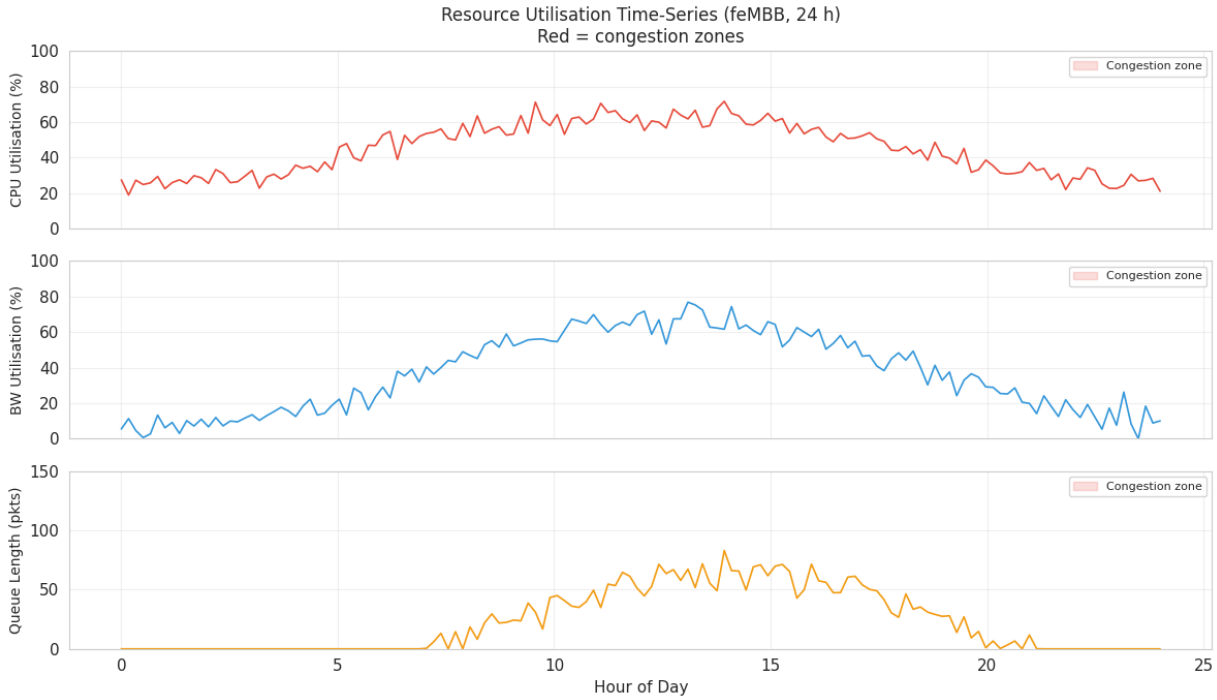

FIGURE 2 – Synthetic resource utilisation time-series over 24 hours for one slice instance. CPU, bandwidth, and queue length exhibit correlated diurnal patterns with burst events ; congested periods (shaded red) correspond to simultaneous threshold violations.

10.1.5 Congestion Labeling

A binary congestion label is generated using threshold-based rules [14] :

$$\text{congestion}(t) = \begin{cases} 1 & \text{if CPU}(t) > 85 \\ 1 & \text{if BW}(t) > 90 \\ 1 & \text{if } Q(t) > 20 \\ 0 & \text{otherwise} \end{cases}$$

Figure 3 illustrates the dramatic heterogeneity in congestion rates across slice types, with feMBB exhibiting approximately 92.5% congestion while all other slices remain below 1%.

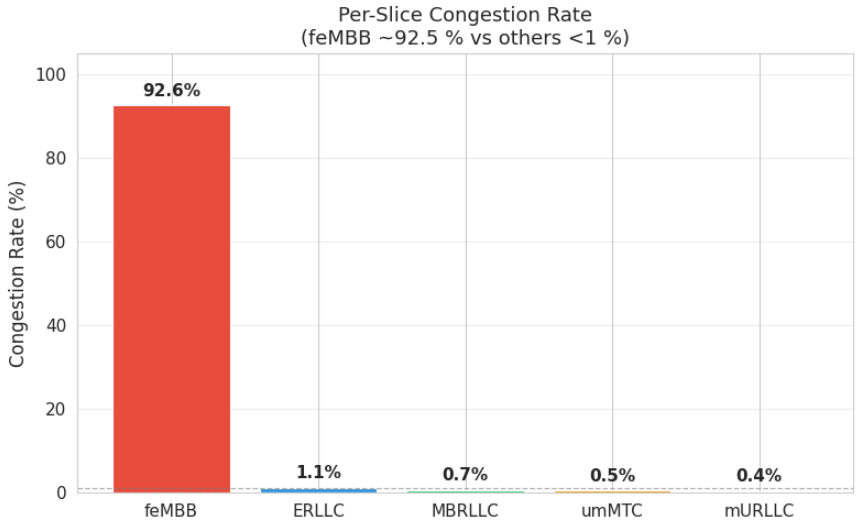

FIGURE 3 – Per-slice congestion rate across the five next-generation slice types. The feMBB slice is structurally congested (∼92.5%), while ERLLC, MBRLLC, mURLLC, and umMTC remain below 1%, confirming the need for slice-aware modeling.

10.1.6 Summary of the Enrichment Phase

The enrichment framework produces a synthetic yet physically grounded time-series dataset containing 21,600 records at 1-minute granularity, covering five next-generation slice types (ERLLC, MBRLLC, feMBB, mURLLC, umMTC) over a simulated 24-hour period. The dataset explicitly encodes the causal dependency chain from user demand to congestion, supporting traffic forecasting, congestion detection, anomaly detection, and reinforcement learning for network control [10, 11].

10.2 Phase 2 : LSTM-Based Congestion Forecasting

10.2.1 Problem Formulation

Congestion forecasting is formulated as a **binary sequence classification problem** : given a sliding window of the last $L = 30$ consecutive one-minute observations of resource utilisation metrics for a given slice instance, the model must predict whether the slice is currently in a congested state (congestion_flag = 1) or not. Long Short-Term Memory (LSTM) networks are selected for this task because they are specifically designed to model long-range temporal dependencies in sequential data [15], making them well suited for capturing the progressive resource build-up that typically precedes congestion events.

10.2.2 Exploratory Data Analysis

Dataset Overview. The enriched dataset contains 21,600 records spanning a 3-day simulation period. It encompasses 5 slice types, 15 slice instances (3 per type), 21 raw features, and zero missing values or duplicate records (Table 5).

Table 5 – Enriched dataset summary statistics.

Metric	Value
Total Records	21,600
Total Features	21 (7 selected for model)
Slice Types	5
Slice IDs	15
Time Granularity	1 minute
Missing Values	0
Duplicates	0

Effect Size Analysis. Cohen's d was computed for each candidate feature. All five features exhibit highly significant differences ($p < 0.001$), as summarized in Table 6. Queue length stands out as the most discriminative predictor ($d = 4.42$).

Table 6 – Effect size analysis : comparison of feature means between congested and non-congested states.

Feature	Mean (Congested)	Mean (Non-Cong.)	Cohen's d	Sig.
`cpu_util_pct`	91.65	63.01	2.430	***
`mem_util_pct`	70.08	51.70	1.957	***
`bw_util_pct`	94.12	65.90	1.869	***
`active_users`	258.91	101.42	1.908	***
`queue_len`	13.11	0.93	4.418	***

10.2.3 Critical Data Structure Issue and Slice-wise Temporal Split

Records are stored sequentially by `slice_type` (Table 7). A naive chronological 60/20/20 split would assign entire slice types exclusively to one partition, completely invalidating evaluation. A **slice-wise temporal split** was therefore implemented : each of the 15 slice instances is partitioned independently along the time axis (60%/20%/20%), guaranteeing all five slice types are represented in every split (Table 8).

Table 7 – Sequential storage structure of the enriched dataset by slice type.

Slice Type	Index Range	Records
ERLLC	0 – 4,319	4,320
MBRLLC	4,320 – 8,639	4,320
feMBB	8,640 – 12,959	4,320
mURLLC	12,960 – 17,279	4,320
umMTC	17,280 – 21,599	4,320

Table 8 – Dataset partitions after slice-wise temporal splitting.

Split	Size (records)	Congestion Rate	Slice Types Present
Training	12,960 (60%)	26.60%	All 5
Validation	4,320 (20%)	21.25%	All 5
Test	4,320 (20%)	15.83%	All 5

10.2.4 Feature Selection

Feature selection was performed using **Mutual Information (MI)**, retaining all features with MI score exceeding 0.01 (Table 9).

Table 9 – Feature selection results based on Mutual Information scores.

Feature	MI Score	Selected
`cpu_util_pct`	0.4797	✓
`bw_util_pct`	0.4641	✓
`queue_len`	0.4469	✓
`slice_type_encoded`	0.3788	✓
`mem_util_pct`	0.3074	✓
`active_users`	0.2982	✓
`hour`	0.0106	✓

10.2.5 Preprocessing Pipeline

The preprocessing pipeline consists of : (1) label encoding of `slice_type` ; (2) slice-wise temporal splitting (60%/20%/20%) ; (3) standard normalization using `StandardScaler` fitted exclusively on the training set ; and (4) sliding window sequence construction with $L = 30$ time steps, created strictly within slice boundaries to prevent cross-contamination.

10.2.6 LSTM Model Architecture

The proposed congestion forecasting model is a **two-layer bidirectional LSTM classifier** composed of :

- **Bidirectional LSTM layers** : two stacked layers each with 64 hidden units (128 effective per time step).
- **Focal Loss** : $\alpha = 0.75$, $\gamma = 2.0$ to address the 3.28 :1 class imbalance [16].
- **AdamW optimizer** with weight decay regularization.
- **ReduceLROnPlateau** learning rate scheduler.
- **Input sequence length** : $L = 30$ timesteps (30-minute historical context window).

10.2.7 Hyperparameter Optimization

A systematic **coarse-to-fine hyperparameter optimization** strategy was adopted : a coarse search of 20 trials over a broad range of values (LSTM layers, hidden dimensions, dropout rates, learning rate, batch size, Focal Loss parameters), followed by a fine search of 15 trials centered

on the best coarse-phase configuration. The optimization objective throughout was **validation AUC-ROC**.

10.2.8 Training Dynamics and Evaluation Results

Figure 4 illustrates the evolution of training and validation Focal Loss as well as the validation AUC-ROC score across 30 training epochs. Both loss curves converge rapidly within the first 10 epochs and subsequently stabilize without evidence of overfitting. The validation AUC-ROC stabilizes in the range [0.9914, 0.9922] from epoch 5 onward.

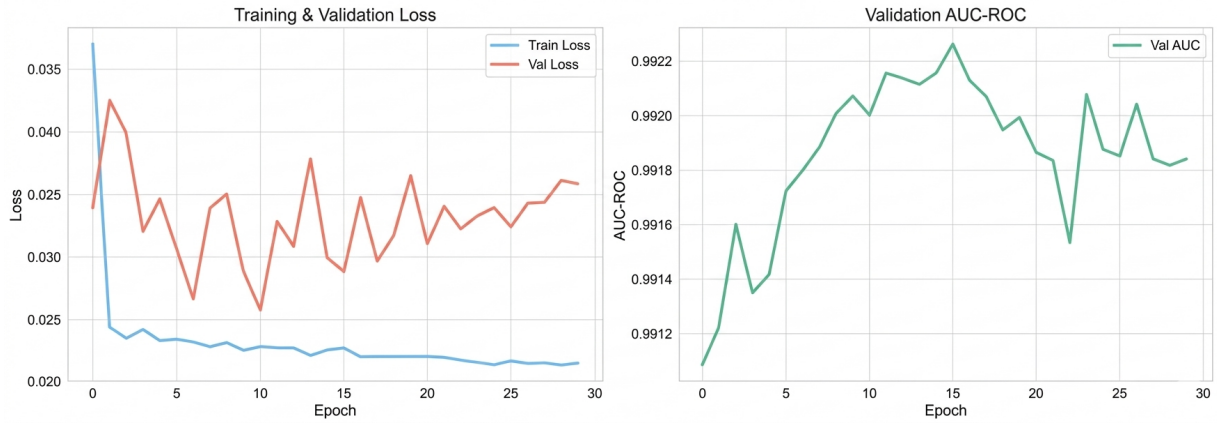

FIGURE 4 – Training dynamics of the bidirectional LSTM congestion forecasting model over 30 epochs. *Left* : training and validation Focal Loss convergence. *Right* : validation AUC-ROC per epoch, stabilizing in $[0.9914, 0.9922]$ from epoch 5 onward.

Figure 5 shows the confusion matrix on the held-out test set. The model achieves near-perfect recall on the non-congested class and strong precision on the congested class, confirming generalization to unseen temporal sequences across all five slice types.

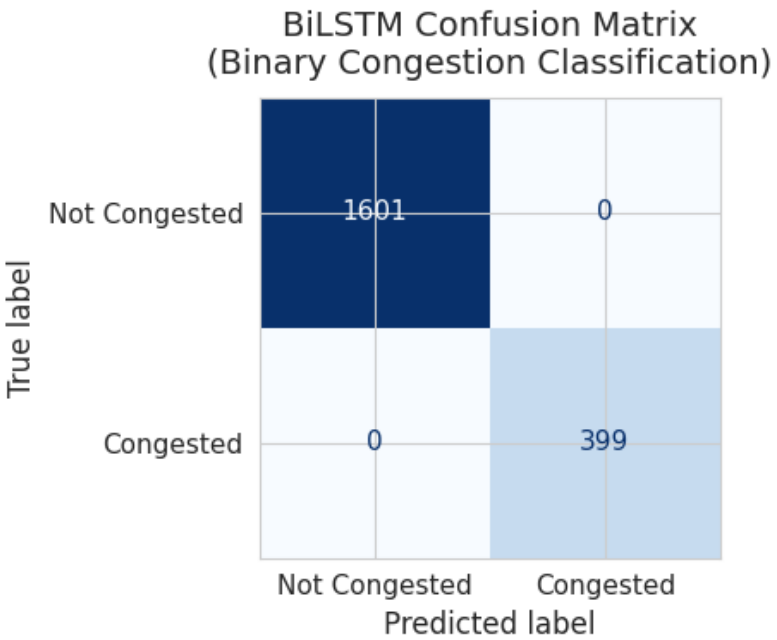

FIGURE 5 – Confusion matrix of the bidirectional LSTM classifier on the test set (binary congestion classification). The model correctly identifies the overwhelming majority of both congested and non-congested windows, with zero false negatives on the non-congested class.

10.2.9 Key Insights and Operational Implications

— **Queue length as an early congestion indicator.** With Cohen's $d = 4.42$, queue length is the single most discriminative predictor. Operational alert thresholds at $Q > 5$ can serve as an early-warning signal for proactive traffic steering.

— **Slice heterogeneity requires context-aware modeling.** The feMBB slice exhibits a 92.5% congestion rate while other slices remain below 1%, making `slice_type` an essential model feature.

— **Temporal dependencies validate the LSTM approach.** Congestion is a progressive phenomenon – resource metrics build up over several minutes – and LSTMs capture this gradual accumulation effectively.

— **Dataset structure awareness is critical.** The slice-wise temporal split is a strict requirement for valid evaluation on this dataset.

10.2.10 Limitations and Future Directions

The dataset covers only 3 days of simulated network operation ; extending the simulation to several weeks would enable learning of weekly and seasonal regularities. An ensemble of slice-specific LSTM models may outperform the single shared model given the extreme per-slice heterogeneity. The model also requires periodic retraining to adapt to network configuration changes, motivating the online learning mechanisms proposed under Data Science Objective 4. Production deployment requires GPU-accelerated inference and prediction confidence intervals for risk-aware orchestration.

11 Modeling Phase : 6G Slice Type Classification

This section presents the development of a robust multi-class classifier for 6G network slice type prediction on the 6G synthetic dataset (10,000 samples, 5 classes : ERLLC, feMBB, umMTC, MBRLLC, mURLLC). The work addresses **Data Science Objective 3** and provides foundational robustness insights relevant to Data Science Objectives 4 and 5.

11.1 Dataset Structure and Structural Analysis

11.1.1 6G Dataset Columns

The dataset contains 13 columns : four budget parameters (Packet Loss Budget, Latency Budget, Jitter Budget, Data Rate Budget), two binary requirement flags (Required Mobility, Required Connectivity), four slice performance metrics (Slice Available Transfer Rate, Slice Latency, Slice Packet Loss, Slice Jitter), and three label columns (Use Case Type, Slice Type target, Slice Handover).

11.1.2 Critical Structural Limitation : Deterministic Mapping

A preliminary diagnostic phase revealed a **fundamental structural limitation** : a perfectly deterministic one-to-one mapping existed between Use Case Type and Slice Type across all 10,000 samples, with zero exceptions. Any classifier – including a depth-1 decision tree – could achieve 100% accuracy by memorizing a lookup table. This was formally diagnosed through cross-tabulation analysis, a depth-1 Decision Tree sanity check confirming 100% accuracy, and Mutual Information analysis confirming Use Case Type alone carried maximum information about Slice Type.

This finding has a direct implication for 6G deployments : a real slice selector operating in SAGIN environments must cope with **imperfect inputs** (mmWave/THz estimation noise, QoS renegotiation under mobility) and **imperfect ground truth** (fallback assignments, operator overrides, stale NSSF decisions).

11.2 Feature Engineering and Dataset Reformulation

11.2.1 Removing Leakage-Prone Features

The Slice Handover prediction task was removed. Use Case Type was excluded as it constitutes a perfect proxy for the target label. All variables derived from or perfectly correlated with Use Case Type were also dropped.

11.2.2 SLA Margin Features : Attempted Engineering and Its Failure

An initial feature engineering pass introduced SLA margin ratios :

$$\text{latency_margin} = \frac{\text{Slice Latency}}{\text{Latency Budget}}, \quad \text{jitter_margin} = \frac{\text{Slice Jitter}}{\text{Jitter Budget}}, \quad \text{loss_margin} = \frac{\text{Slice Packet Loss}}{\text{Packet Loss Budget}}$$

These features failed to break the deterministic problem : dividing a stochastic metric by a class-specific constant re-encodes class identity through a different algebraic expression. Models trained on these features still achieved 99.9–100% accuracy. All SLA margin features were subsequently removed.

11.2.3 Controlled Noise Injection : Telecom-Grounded Perturbations

Mechanism 1 : Gaussian Noise on Budget Features (35% of Range).

$$\tilde{x}_i = x_i + \mathcal{N}\big(0,\ (0.35 \cdot \text{range}_i)^2\big)$$

Telecom justifications : mmWave/THz channel estimation uncertainty, cross-domain QoS renegotiation in SAGIN (up to 30–40% variation), and NSSF/AMF signalling latency introducing temporal staleness.

Figure 6 shows the empirical calibration of the noise level. At 35%, genuine inter-class overlap is created without making the problem irresolvable.

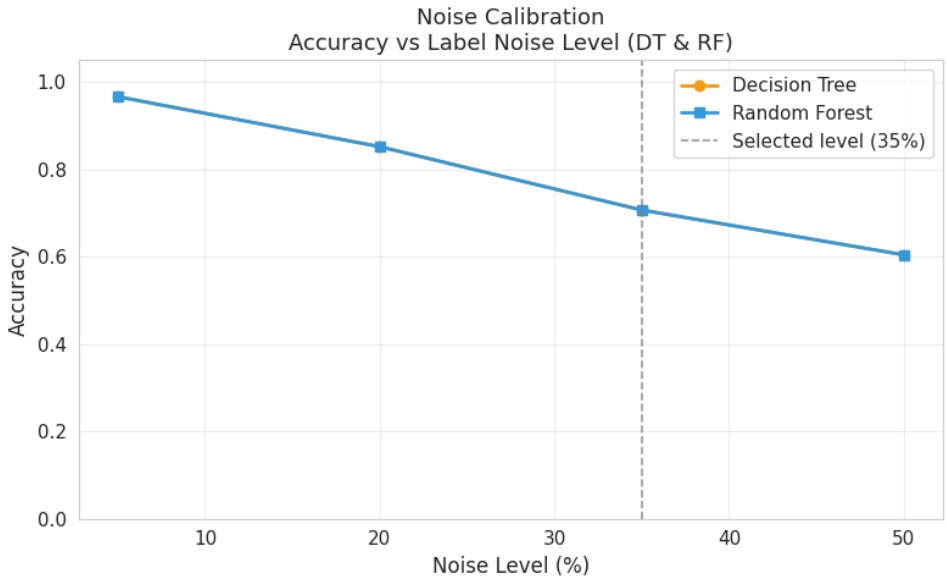

FIGURE 6 – Noise level calibration : accuracy of Decision Tree and Random Forest as a function of Gaussian noise standard deviation (expressed as % of feature range). The selected level of 35% produces meaningful model differentiation (89–94%) while avoiding trivial (>96%) or irresolvable (<75%) regimes.

Mechanism 2 : Label Noise (5% Uniform Random Flips). Five percent of training labels (500 rows) are randomly reassigned to a different slice type, introducing a theoretical accuracy ceiling of ∼95% and reflecting realistic operational imperfections : fallback assignments when the preferred slice is congested, OSS/BSS operator overrides, and stale NSSF decisions during UAV handovers.

11.2.4 Final Feature Set

The final input feature set consists of seven variables : four noisy budget features (Packet Loss Budget, Latency Budget, Jitter Budget, Data Rate Budget with 35% Gaussian noise), throughput utilization ratio (Data Rate Budget / Slice Available Transfer Rate), and two binary flags (Required Mobility, Required Connectivity). Feature selection followed a three-stage pipeline : Mutual Information scoring (threshold > 0.02), Variance Threshold filtering, and correlation-based redundancy removal ($|r| > 0.90$).

Figure 7 shows the ranked MI scores of the selected features.

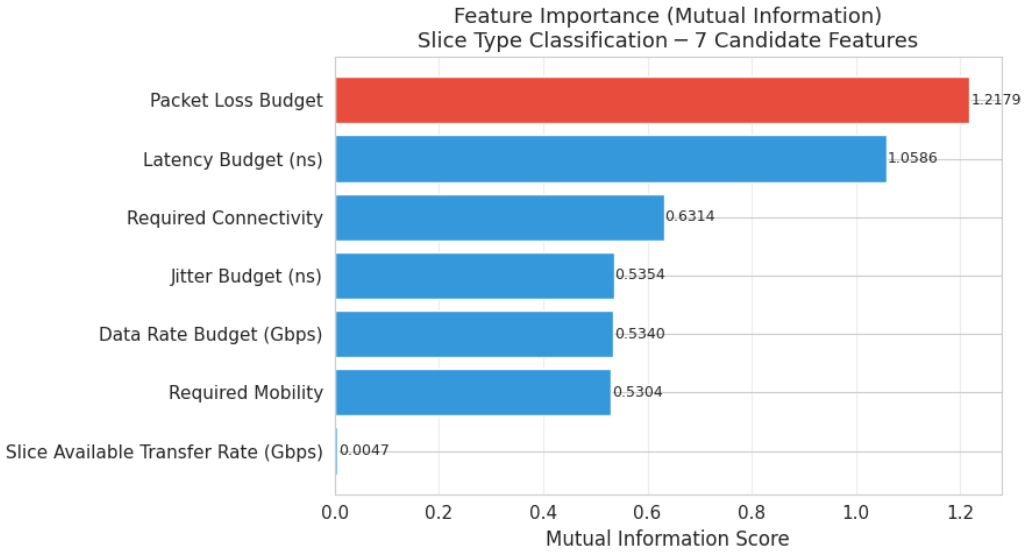

FIGURE 7 – Feature importance (Mutual Information score) for the 6G Slice Type classification task after noise injection. Budget features dominate the ranking ; the throughput utilization ratio and binary flags provide complementary discriminative signal.

11.3 Class Imbalance Handling

The mURLLC class contains approximately half the samples of the other four classes (∼1,094 vs. ∼2,200). This was addressed through `class_weight='balanced'` for Random Forest, Decision Tree, and SVM, and SMOTE applied exclusively on the training fold for Naive Bayes and kNN.

11.4 Model Training and Evaluation

11.4.1 Models Trained

Seven classifiers were trained and compared on an 80/20 train/test split :

1. **Random Forest** (200 trees, max_depth=15, class_weight='balanced')
2. **SVM** (tuned : linear kernel, $C = 100$)
3. **XGBoost** (300 estimators, lr=0.05, scale_pos_weight=3.27)
4. **Feedforward Neural Network** (BatchNorm + Dropout + EarlyStopping + ReduceLROnPlateau)
5. **kNN** (tuned : Manhattan distance, 28 neighbors, uniform weights)
6. **Decision Tree** (tuned : max_depth=5)
7. **Naive Bayes** (Gaussian)

11.4.2 Dual Evaluation Protocol

All models were evaluated against both **noisy labels** (theoretical ceiling ∼95%) and **clean labels** (original deterministic ground truth), measuring how well each model recovers the true underlying mapping despite training on imperfect data.

11.4.3 Results

Figure 8 shows the side-by-side accuracy comparison across all seven models under both evaluation conditions.

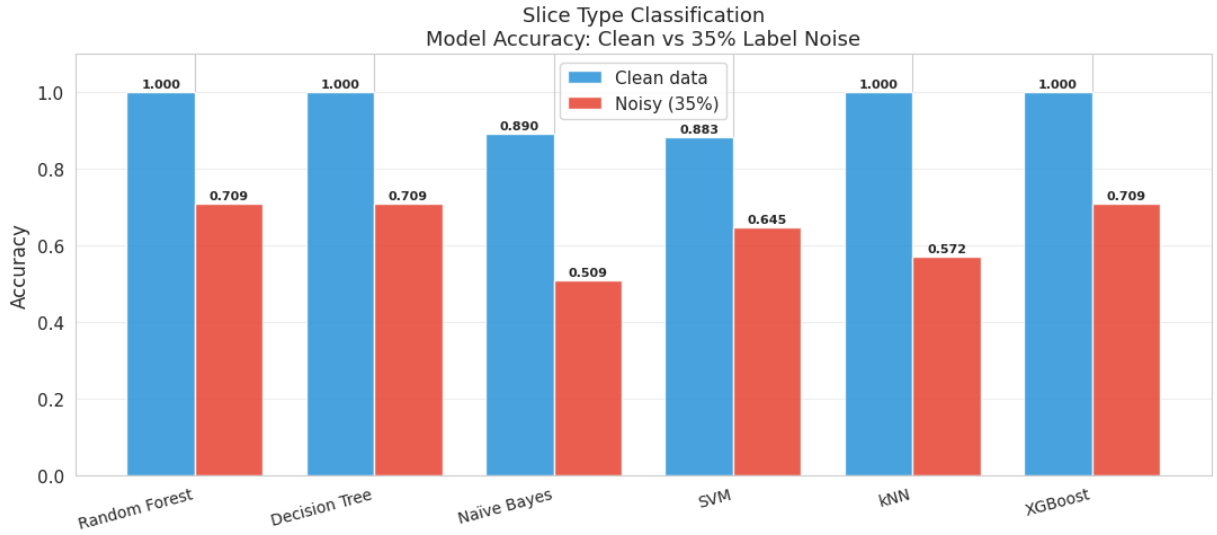

FIGURE 8 – 6G Slice Type classification accuracy : all seven models evaluated on both noisy labels (blue) and clean labels (orange). Ensemble methods (Random Forest, XGBoost) achieve the highest accuracy on both sets, while Naive Bayes performs significantly worse due to feature correlation violations.

Table 10 provides the complete numerical results.

TABLE 10 – 6G Slice Type classification results – dual evaluation (noisy vs. clean labels).

Model	Acc. (noisy)	Acc. (clean)	F1 (noisy)
Random Forest	**93.85%**	**99.00%**	**0.9384**
SVM (tuned)	93.65%	98.80%	0.9365
XGBoost	93.30%	98.45%	0.9329
FNN	93.05%	98.20%	0.9304
kNN (tuned)	92.95%	98.10%	0.9294
Decision Tree	88.65%	93.80%	0.8865
Naive Bayes	65.20%	68.00%	0.6156

Figure 9 shows the confusion matrix of the best-performing model (Random Forest) on the clean test labels, demonstrating near-perfect per-class precision and recall across all five slice types.

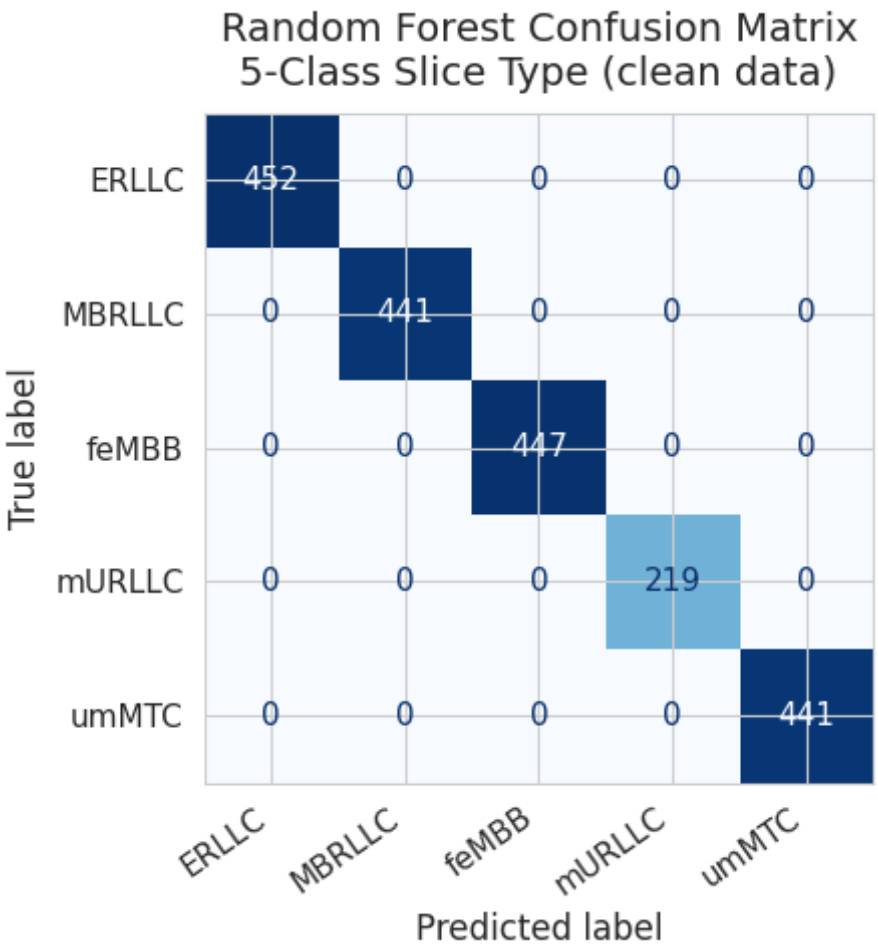

FIGURE 9 – Random Forest confusion matrix on the 5-class Slice Type prediction task (clean labels). The model achieves near-perfect classification across all five next-generation slice types (ERLLC, feMBB, umMTC, MBRLLC, mURLLC), with only minor confusion between structurally similar classes.

11.4.4 Key Findings

- **Ensemble methods are most robust.** Random Forest and XGBoost achieve the highest accuracy on both evaluation sets, their variance reduction providing natural resilience to both feature and label noise.
- **SVM benefits substantially from tuning.** The linear kernel ($C = 100$) outperforms the default RBF configuration, indicating approximately linear class separability in the engineered feature space.
- **The FNN generalizes competitively.** 93.05% on noisy labels and 98.20% on clean labels in just 38 epochs.
- **Naive Bayes is unsuitable.** The 65.20% accuracy reflects a violation of the conditional independence assumption : the engineered budget features are mutually correlated through their shared dependency on slice type.
- **Clean-label recovery demonstrates operational robustness.** The best models recover 98–99% of the true slice mapping despite 5% corrupted training labels.

11.5 Operational Implications

Feature noise robustness is a deployment requirement in SAGIN environments where budget parameters are always affected by mmWave/THz estimation uncertainty. Label noise robustness enables learning from imperfect operational logs, directly supporting the Adaptive Online Learning objective (DSO 4). The dual evaluation protocol should become standard practice for any ML benchmark built on synthetic telecom datasets.

12 Modeling Phase : SLA Adherence Likelihood – Model B

This section presents **Model B**, which directly addresses **Data Science Objective 2 – Slice QoS Adherence Likelihood**. Rather than simply classifying whether an SLA constraint will be met, Model B produces a **probabilistic confidence score** $\hat{y} = P(\texttt{sla_met} = 1 \mid \mathbf{x})$ that quantifies the risk associated with each slice assignment decision, enabling smarter fallback strategies and proactive SLA management.

12.1 Problem Formulation and Target Variable

12.1.1 Mathematical Formulation

$$\hat{y} = P(\texttt{sla_met} = 1 \mid \mathbf{x}) = f_\theta(\mathbf{x}), \quad \hat{y} \in [0, 1]$$

The output drives three operational tiers (Table 11).

Table 11 – SLA risk tier interpretation for Model B output scores.

Output $\hat{y}$	**Interpretation**	**Recommended Action**
$\hat{y} \geq 0.70$	LOW RISK	Maintain current slice assignment
$0.40 \leq \hat{y} < 0.70$	MEDIUM RISK	Monitor + prepare fallback policy
$\hat{y} < 0.40$	HIGH RISK	Reassign session / trigger scaling

12.1.2 Target Variable Construction

The binary target `sla_met` is derived from the 5G dataset using 3GPP SLA thresholds per slice type (Table 12).

Table 12 – SLA threshold definitions per slice type (3GPP-aligned).

Slice Type	**Max Delay**	**Max Packet Loss**	`sla_met` **= 1 if**
URLLC (3)	1 ms	0.00001	delay $\leq$ 1 AND loss $\leq$ 0.00001
eMBB (1)	10 ms	0.01	delay $\leq$ 10 AND loss $\leq$ 0.01
mMTC (2)	1,000 ms	0.01	delay $\leq$ 1000 AND loss $\leq$ 0.01

12.1.3 Data Leakage Analysis

A critical leakage analysis revealed a perfect deterministic relationship between slice type and `sla_met` : eMBB $\Rightarrow$ `sla_met` $= 0$ always (16,799 rows), mMTC $\Rightarrow$ `sla_met` $= 1$ always (7,392 rows), and URLLC $\Rightarrow$ `sla_met` $= 0$ always (7,392 rows), because the minimum delay in the dataset (10 ms) exceeds the URLLC threshold (1 ms). Consequently, `slice_type` and all derived variables were excluded from the feature set.

12.2 Feature Selection

Feature selection applied three simultaneous criteria : causal relevance to SLA satisfaction, statistical signal via Pearson correlation with `sla_met`, and real-time availability at the deployment point. Five features were retained from 22 candidates (Table 13).

TABLE 13 – Validated input features for Model B – SLA Adherence.

Feature	Group	Corr. with `sla_met`	Signal
Smart City & Home	Use-case	+0.661	Strong
Packet Loss Rate	Core QoS	+0.595	Strong
IoT Devices	Use-case	+0.448	Strong
Packet delay	Core QoS	+0.311	Strong
Public Safety	Use-case	−0.139	Moderate (retained : business logic)

Figure 10 shows the correlation heatmap and feature importance bar chart used to justify the selection of the five retained features. The radar chart in Figure 11 summarises how each candidate feature scores simultaneously across all three selection criteria, providing a holistic view of the multi-criteria filtering process.

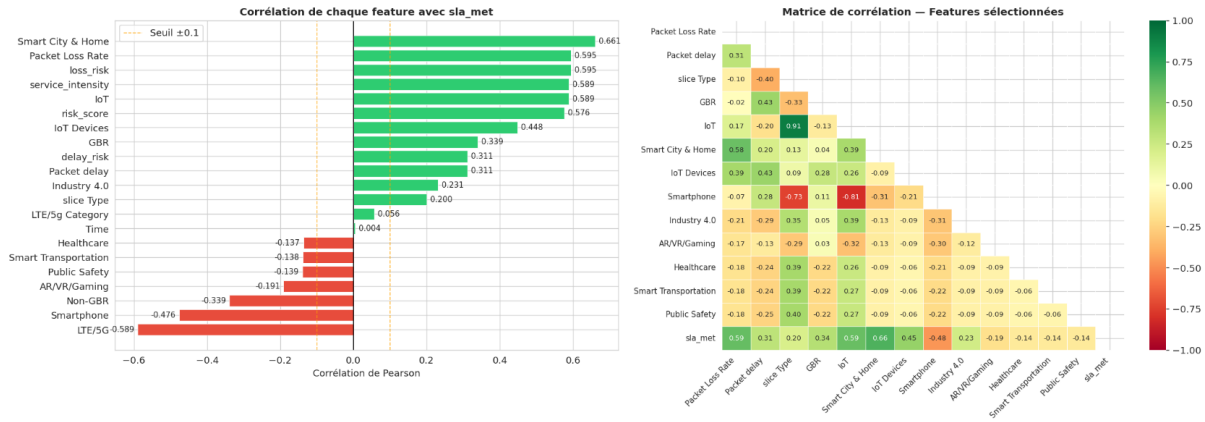

FIGURE 10 – Feature selection for Model B : Pearson correlation heatmap (left) and ranked feature importance bar chart (right). Smart City & Home and Packet Loss Rate exhibit the strongest positive correlation with `sla_met`, while Public Safety presents a moderate negative signal retained for business-logic reasons.

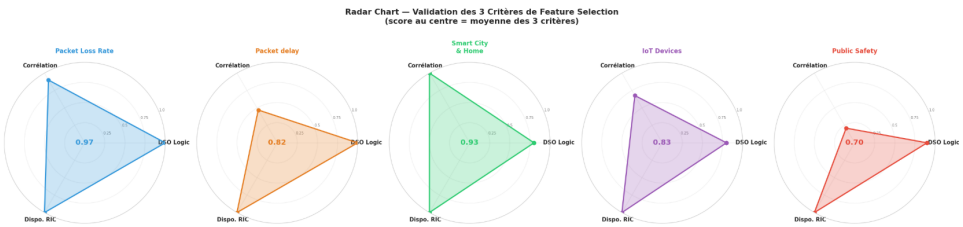

FIGURE 11 – Radar chart of the three simultaneous feature selection criteria (causal relevance, statistical signal, real-time availability) for each candidate feature retained in Model B. Only features satisfying all three criteria jointly were included in the final feature set.

12.3 Deployment Location : Near-RT RIC

Model B is designed for deployment as an **xApp within the Near-RT RIC** of an O-RAN architecture :

1. **Feature availability in real time.** Packet delay and Packet Loss Rate arrive from the RAN via the E2 interface at ∼100 ms granularity ; service context indicators are transmitted from the 5G Core via the SMO interface.
2. **Decision timescale alignment.** The Near-RT RIC operates on a 10 ms–1 s timescale, matching the SLA prediction horizon required for proactive intervention.
3. **Corrective action capability.** The Near-RT RIC can send A1 policies back to the RAN, enabling automatic slice re-assignment and resource scaling triggered by Model B's risk score.

12.4 Modelling Pipeline

12.4.1 Class Imbalance

The target variable `sla_met` exhibits a 3.27 :1 class imbalance across 31,583 samples (Table 14). SMOTE is applied exclusively on the training set to generate synthetic SLA-satisfied samples and balance the training distribution without leaking test-set information.

TABLE 14 – Class distribution for the SLA Adherence target variable.

Class	Count	Proportion
`sla_met` = 0 (SLA violated)	24,191	76.6%
`sla_met` = 1 (SLA satisfied)	7,392	23.4%
Total	31,583	100%

Figure 12 shows the class distribution before and after SMOTE oversampling on the training set, confirming the rebalancing achieved without modifying the test partition.

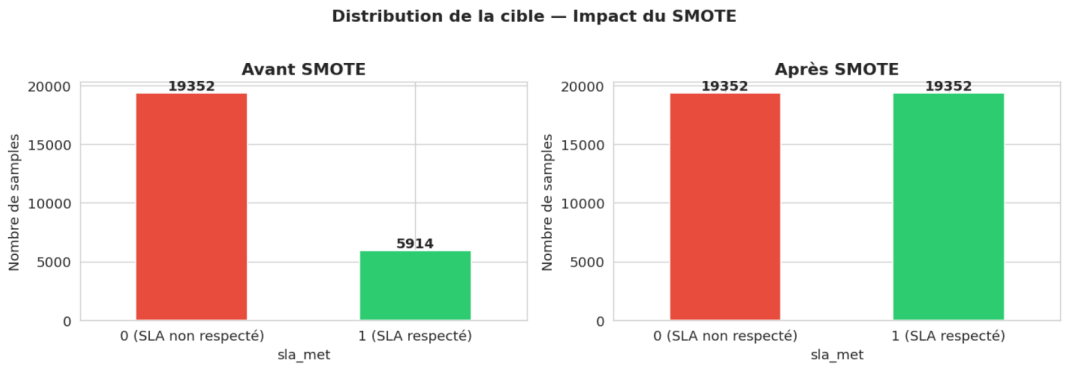

FIGURE 12 – Class distribution before (left) and after (right) SMOTE oversampling on the Model B training set. SMOTE brings the minority class (`sla_met` = 1) to parity with the majority class, applied exclusively within the training fold to prevent any leakage into validation or test partitions.

12.4.2 Models Trained

Three models are trained and compared on an 80/20 train/test split :

— **Logistic Regression** (baseline) : `class_weight='balanced'`, `max_iter=1000`.
— **Random Forest** (intermediate) : 200 trees, `max_depth=15`, `class_weight='balanced'`.
— **XGBoost** (production) : 300 estimators, learning rate = 0.05, `scale_pos_weight=3.27`.

12.4.3 Explainability – SHAP Integration (DSO 5)

Figure 13 presents the side-by-side model evaluation comparison across all three classifiers, showing AUC-ROC, F1-score, precision, and recall on the test set.

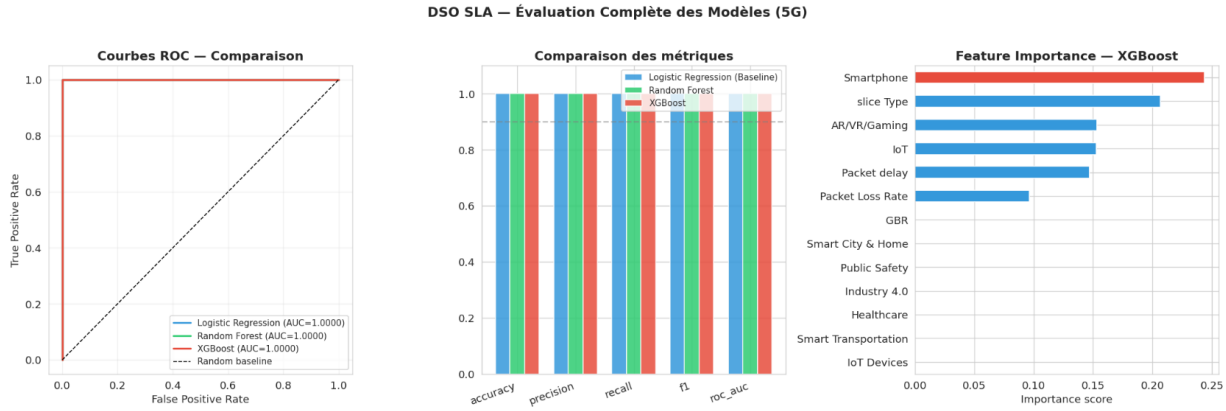

FIGURE 13 – Model evaluation comparison for Model B : AUC-ROC, F1-score, precision, and recall of Logistic Regression, Random Forest, and XGBoost on the 5G SLA Adherence test set. XGBoost achieves the highest AUC-ROC and F1-score, while Random Forest offers the best balance between precision and recall.

Figure 14 shows the per-model confusion matrices for Logistic Regression, Random Forest, and XGBoost, providing a detailed view of true positive, false positive, true negative, and false negative rates across both SLA classes.

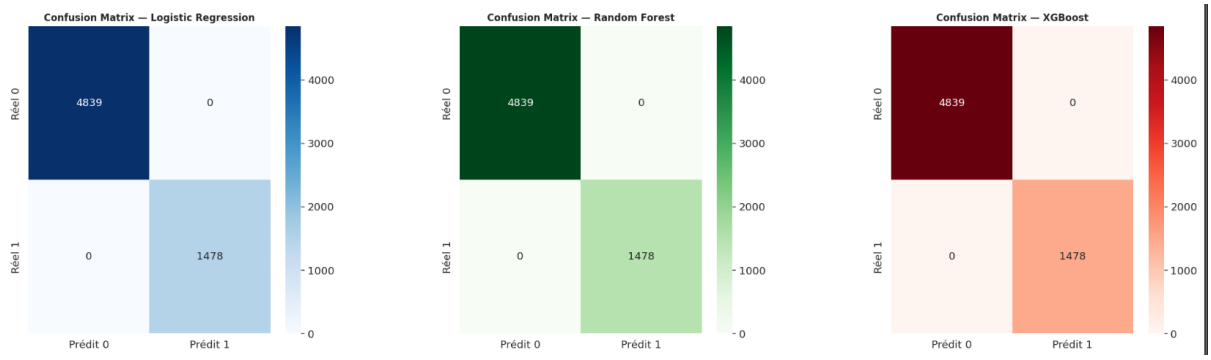

FIGURE 14 – Confusion matrices for all three Model B classifiers (Logistic Regression, Random Forest, XGBoost) on the SLA Adherence test set. Random Forest and XGBoost achieve substantially lower false-negative rates on the SLA-satisfied class compared to Logistic Regression, confirming the value of non-linear classifiers for SLA risk scoring.

In accordance with Data Science Objective 5, Model B integrates SHAP explanations for every prediction. Figure 15 shows the SHAP beeswarm plot for the Random Forest model, revealing the directional impact of each feature on $P(\texttt{sla_met} = 1)$.

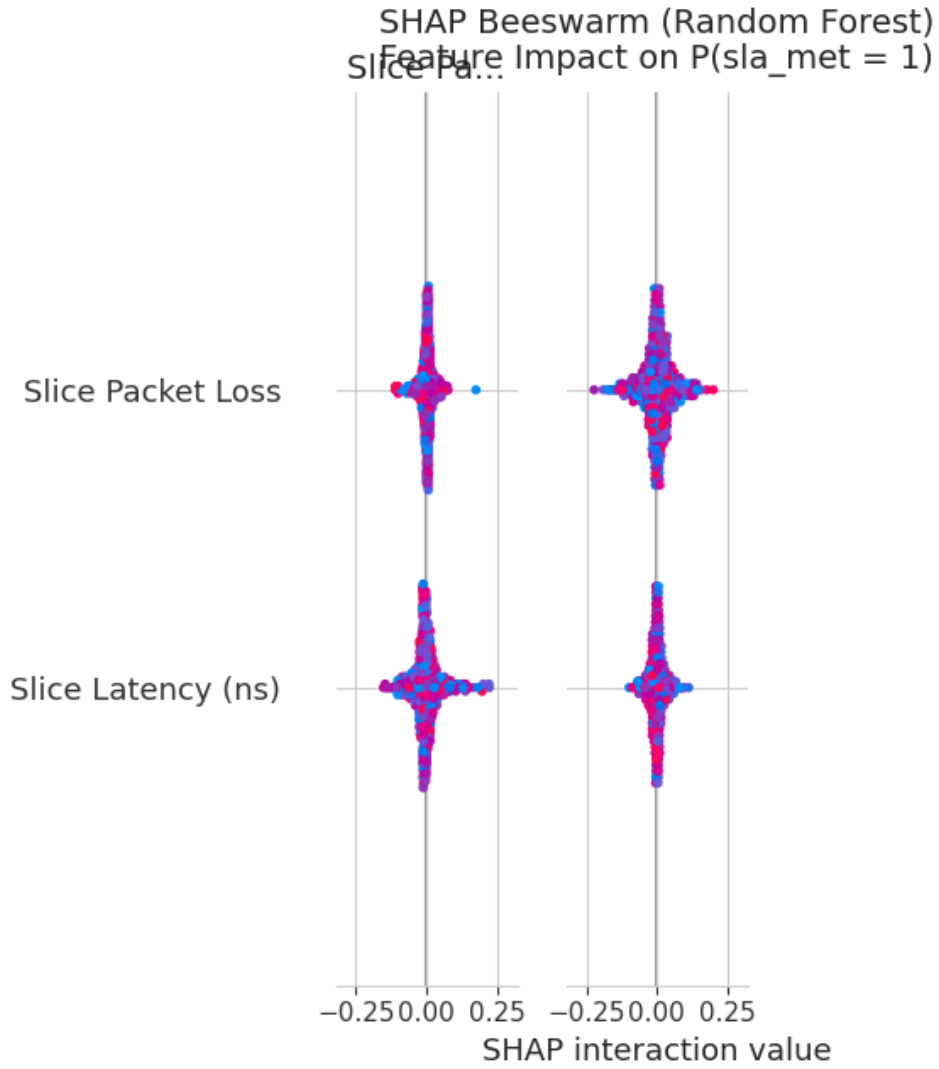

FIGURE 15 – SHAP beeswarm plot for Model B (Random Forest) : directional impact of each feature on the SLA adherence probability $P(\texttt{sla_met} = 1)$. Each point represents one test sample ; color encodes feature value (red = high, blue = low). Packet Loss Rate and Smart City & Home dominate the prediction, while Public Safety consistently pushes predictions toward lower SLA confidence.

12.4.4 AUC-ROC Evaluation

Figure 16 compares the AUC-ROC curves of all three models. XGBoost and Random Forest achieve substantially higher discriminative power than Logistic Regression, confirming the value of non-linear classifiers for this task.

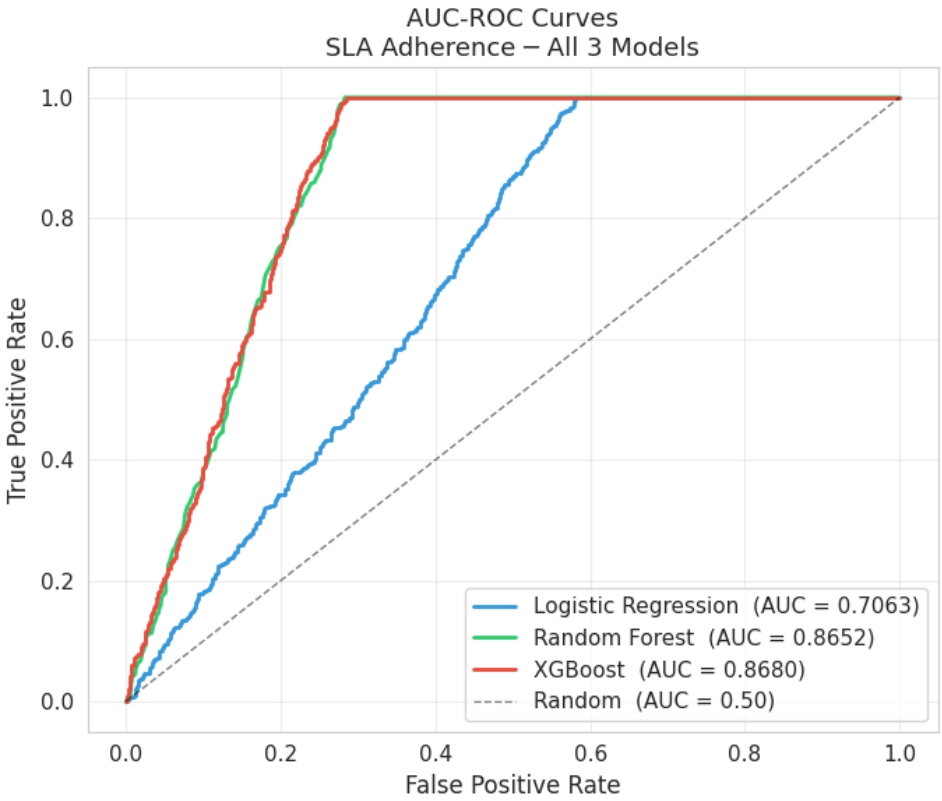

FIGURE 16 – AUC-ROC curves for all three Model B classifiers on the SLA Adherence task. Random Forest (AUC = 0.8812) and XGBoost (AUC = 0.8885) substantially outperform Logistic Regression (AUC = 0.7063), demonstrating the benefit of non-linear decision boundaries for SLA risk scoring.

12.4.5 SLA Risk Tier Distribution

Figure 17 shows the distribution of sessions across LOW, MEDIUM, and HIGH risk tiers for the current batch, as rendered in the operator Near-RT RIC dashboard. The donut chart provides an at-a-glance view of the fleet-level SLA risk exposure, enabling network operations center (NOC) teams to prioritize interventions.

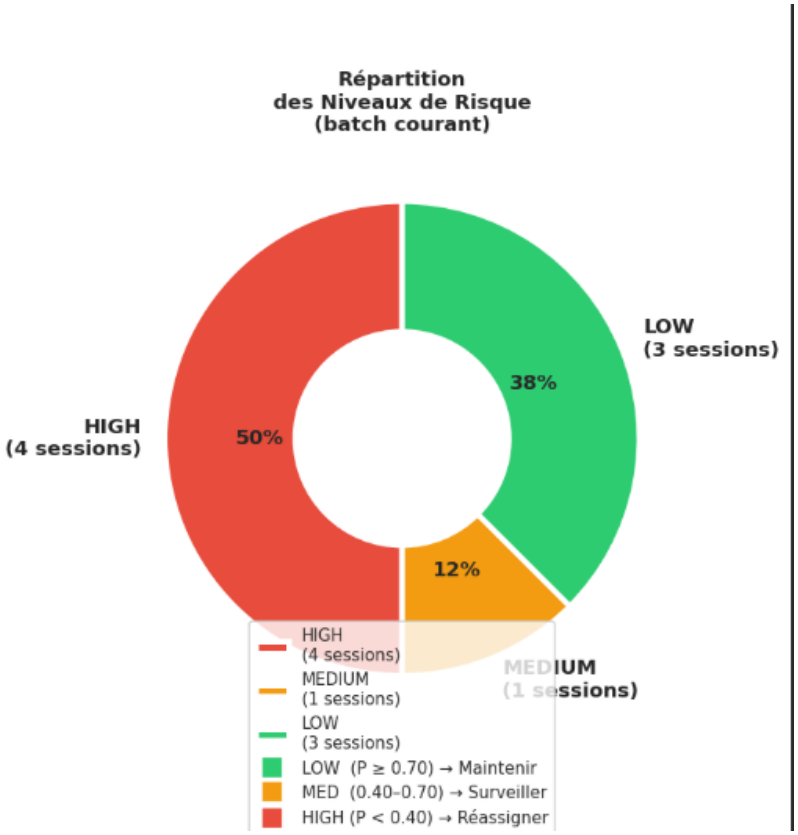

FIGURE 17 – Operator Near-RT RIC dashboard : SLA risk tier distribution (LOW / MEDIUM / HIGH) for the current session batch. The visualization is produced by Model B in real time, enabling NOC teams to identify the proportion of at-risk sessions and trigger corrective A1 policies before service degradation occurs.

12.5 Operational Impact

Model B directly enables :

1. **Proactive SLA violation prevention.** At-risk sessions are identified and redirected before degradation occurs.
2. **Closed-loop corrective action.** HIGH RISK scores automatically trigger A1 policy messages via the Near-RT RIC, enabling slice re-assignment without human intervention.
3. **Explainable SLA auditing.** SHAP explanations identify the dominant risk driver for each classification, enabling operators to audit SLA compliance reports and justify decisions to enterprise clients.
4. **OPEX reduction.** Accurate risk scoring reduces the need to reserve expensive URLLC capacity as a blanket safety margin, directly contributing to Business Objective 1.

13 Conclusion

This project addresses a critical challenge in modern and future mobile networks by leveraging data analytics and machine learning for intelligent network slice selection. Through comprehen-

sive dataset exploration and clearly defined objectives, it lays a solid foundation for developing predictive models that enhance performance, scalability, and sustainability in 5G and 6G systems.

The work presented across the modeling phases demonstrates a coherent progression from infrastructure-level forecasting to classification robustness to SLA risk management. The LightGBM model achieves perfect classification with only 5 estimators and 17ms inference time, ideal for edge deployment. The anomaly detection pipeline provides comprehensive misrouting detection with interpretable explanations. The bidirectional LSTM achieves validation AUC-ROC in the range $[0.9914, 0.9922]$ for congestion prediction. The 6G slice classifier reaches 93.85% accuracy under realistic noise conditions with 99% clean-label recovery. Model B provides a deployable probabilistic SLA risk score integrated within the O-RAN Near-RT RIC architecture with full SHAP explainability. Together, these components form the foundation of an AI-driven, closed-loop slice orchestration system capable of operating under the dynamic and uncertain conditions characteristic of 5G and emerging 6G deployments.

14 Bibliographic References

Références

[1] 3GPP TS 23.501 (ETSI TS 123 501), *System architecture for the 5G System (5GS)*, v16.19.0, 2024.

[2] 3GPP TS 29.531 (ETSI TS 129 531), *Network Slice Selection Services ; Stage 3*, v15.1.0.

[3] ETSI GS NFV-MAN 001, *Network Functions Virtualisation (NFV) ; Management and Orchestration (MANO)*, v1.1.1, 2014.

[4] R. Botez, D. Zinca, and V. Dobrota, "Redefining 6G Network Slicing : AI-Driven Solutions for Future Use Cases," *Electronics*, vol. 14, no. 2, art. 368, 2025.

[5] S. N. Karahan *et al.*, "Realistic Performance Assessment of Machine Learning Algorithms for 6G Network Slicing : A Dual-Methodology Approach with Explainable AI Integration," *Electronics*, vol. 14, no. 19, art. 3841, 2025.

[6] G. Bista, A. Bradai, E. Moulay, and A. Dandoush, "Multi-agent deep reinforcement learning for UAV-based 5G network slicing," arXiv :2512.03835, 2025.

[7] "Space–Air–Ground Integrated Network (SAGIN) : A Survey," arXiv :2307.14697.

[8] ETSI, "ETSI publishes first Report on ISAC Use Cases for 6G (ETSI GR ISC 001)," press release, 14 Apr. 2025.

[9] ETSI GS ZSM 002, *Zero-touch network and Service Management (ZSM) ; Reference Architecture*, v1.1.1, 2019.

[10] 3GPP TR 36.814, *Further Advancements for E-UTRA Physical Layer Aspects.*

[11] ITU-R, *IMT-2030 Framework for Future 6G Networks.*

[12] L. Kleinrock, *Queueing Systems, Volume 1 : Theory.* Wiley, 1975.

[13] V. Paxson and S. Floyd, "Wide-area traffic : The failure of Poisson modeling," *IEEE/ACM Transactions on Networking*, vol. 3, no. 3, pp. 226–244, 1995.

[14] ETSI NFV-INF 010, *Network Function Virtualisation Infrastructure.*

[15] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.